BIG DATA ANALYTICS LAB

Experiments 1–9: Code, Commands, and Visualization Dataset

Prepared from the uploaded CEPEL5014 Big Data Analytics Lab experiment list (Academic Year 2026–27).

Google Colab notes: Python sections are designed for Colab. Hadoop, HDFS, Hive, and HBase commands require the corresponding services to be configured. MongoDB code requires a reachable server or Atlas URI. The visualization experiment below includes its own small dataset inside the code, so no separate CSV file is needed for that part.

Contents

1. Hadoop HDFS practical
2. Hadoop MapReduce word count
3. MongoDB and HBase NoSQL commands
4. Hive database and descriptive statistics
5. DGIM, Bloom Filter, and Flajolet–Martin
6. Social Network Analysis / Community Detection
7. Data Visualization (embedded sample dataset)
8. PySpark EDA and HITS
9. Sales Analytics mini-project

How to use this PDF

1. Open Google Colab. 2. Create a new notebook. 3. Copy each experiment's code cell into a code cell. 4. Run cells from top to bottom. 5. For the actual Hadoop/Hive/HBase commands, run them in a configured environment that has those services installed and running.

The common dataset cell

Common dataset — 500 synthetic sales records

This generates bda_sales_500.csv with dates in dd-mm-yyyy format from 2011–2012 and floating-point sales measures. Experiments using the common dataset can run this first.

```
import sys, subprocess, importlib.util, os, random, math, json, re
from datetime import datetime, timedelta
import numpy as np
import pandas as pd

SEED = 42
rng = np.random.default_rng(SEED)
n = 500
dates = pd.to_datetime(rng.choice(pd.date_range("2011-01-01", "2012-12-31", freq="D"), size=n))
regions = rng.choice(["North", "South", "East", "West"], size=n)
products = rng.choice(["Laptop", "Phone", "Tablet", "Monitor", "Keyboard"], size=n)
units = rng.integers(1, 30, size=n)
unit_price = np.round(rng.uniform(250.0, 25000.0, size=n), 2)
discount = np.round(rng.uniform(0.00, 0.25, size=n), 3)
revenue = np.round(units * unit_price * (1 - discount), 2)
cost = np.round(revenue * rng.uniform(0.55, 0.88, size=n), 2)
sales = pd.DataFrame({
    "record_id": np.arange(1, n+1),
    "date": pd.Series(dates).dt.strftime("%d-%m-%Y"),
    "region": regions,
    "product": products,
    "units_sold": units,
    "unit_price": unit_price,
    "discount_rate": discount,
    "revenue": revenue,
    "cost": cost,
})
sales["profit"] = np.round(sales["revenue"] - sales["cost"], 2)
sales.to_csv("bda_sales_500.csv", index=False)
sales.to_json("bda_sales_500.json", orient="records", indent=2)
print("Dataset rows:", len(sales))
display(sales.head(10))
print("CSV saved to:", os.path.abspath("bda_sales_500.csv"))
print(sales.dtypes)
```

Experiment 1 — Hadoop HDFS practical

Aim: Learn HDFS basics, copy files to/from HDFS, list, display, move, and delete files.

A. Colab-friendly local file-system demonstration

This runs without Hadoop and demonstrates the same file operations locally. It is **not** proof that HDFS is running.

Code cell

```
import os, shutil
os.makedirs("/content/bda_input", exist_ok=True)
shutil.copy("bda_sales_500.csv", "/content/bda_input/bda_sales_500.csv")
print("Files:", os.listdir("/content/bda_input"))
print("\nFirst five lines:")
with open("/content/bda_input/bda_sales_500.csv", encoding="utf-8") as f:
    for _ in range(5): print(f.readline().rstrip())
os.makedirs("/content/bda_archive", exist_ok=True)
shutil.move("/content/bda_input/bda_sales_500.csv", "/content/bda_archive/bda_sales_500.csv")
print("After move:", os.listdir("/content/bda_archive"))
os.remove("/content/bda_archive/bda_sales_500.csv")
print("After delete:", os.listdir("/content/bda_archive"))
```

B. Actual HDFS commands (run only where Hadoop is installed and NameNode/DataNode are running)

Upload the CSV to Colab first if using another environment, then adjust the local path. Commands are shell commands, not Python.

hdfs dfs -mkdir -p /user/$USER/bda

hdfs dfs -put -f bda_sales_500.csv /user/$USER/bda/

hdfs dfs -ls /user/$USER/bda

hdfs dfs -cat /user/$USER/bda/bda_sales_500.csv | head

hdfs dfs -get /user/$USER/bda/bda_sales_500.csv ./downloaded_sales.csv

hdfs dfs -mv /user/$USER/bda/bda_sales_500.csv /user/$USER/bda/sales_moved.csv

hdfs dfs -rm /user/$USER/bda/sales_moved.csv

hdfs dfs -rmdir /user/$USER/bda

Hadoop ecosystem overview: HDFS stores distributed files; YARN manages cluster resources; MapReduce processes data in parallel; Hive offers SQL-like queries; HBase is a column-family NoSQL database; Pig uses a data-flow language.

Experiment 2 — MapReduce Word Count

A. Runnable MapReduce pattern in Python (mapper → shuffle/group → reducer)

Code cell

```
from collections import defaultdict
text_data = """big data analytics big data
hadoop hive mongodb hadoop
analytics data visualization hive"""

# Mapper: emit (word, 1)
mapped = []
for line in text_data.lower().splitlines():
    for word in re.findall(r"[a-z0-9]+", line):
        mapped.append((word, 1))

# Shuffle/sort: group values by key
grouped = defaultdict(list)
for key, value in mapped:
    grouped[key].append(value)

# Reducer: sum values for each key
word_counts = {key: sum(values) for key, values in sorted(grouped.items())}
print("Word count output:")
for word, count in word_counts.items():
    print(word, count)
```

B. Hadoop Streaming files and commands

Save the following as `mapper.py` and `reducer.py` on a machine with Hadoop Streaming installed. This is the actual Hadoop job template; the cluster and streaming JAR must be configured.

Code cell

```
mapper_source = r'''#!/usr/bin/env python3
import sys, re
for line in sys.stdin:
    for word in re.findall(r"[a-z0-9]+", line.lower()):
        print(f"{word}\t1")
'''
reducer_source = r'''#!/usr/bin/env python3
import sys
current = None
total = 0
for line in sys.stdin:
    word, count = line.rstrip("\n").split("\t", 1)
    count = int(count)
    if current is not None and word != current:
        print(f"{current}\t{total}")
        total = 0
    current = word
    total += count
if current is not None:
    print(f"{current}\t{total}")
'''
open("mapper.py", "w").write(mapper_source)
open("reducer.py", "w").write(reducer_source)
print("Created mapper.py and reducer.py")
```

Example command on a configured Hadoop machine (replace the JAR path with the installed version):

chmod +x mapper.py reducer.py

hdfs dfs -mkdir -p /user/$USER/wordcount_input

hdfs dfs -put -f input.txt /user/$USER/wordcount_input/

hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \

-input /user/$USER/wordcount_input \

-output /user/$USER/wordcount_output \

-mapper mapper.py -reducer reducer.py \

-file mapper.py -file reducer.py

hdfs dfs -cat /user/$USER/wordcount_output/part-*

Experiment 3 — MongoDB / HBase NoSQL commands

A. MongoDB using PyMongo

A MongoDB server is required. Use MongoDB Atlas or your own server and store the connection URI in Colab Secrets/environment variable `MONGODB_URI`. Do not publish your password or URI.

Code cell

```
# Install driver (run once)
%pip -q install pymongo
```

Code cell

```
# Optional: execute only after setting MONGODB_URI to a reachable MongoDB connection string.
# import os
# from pymongo import MongoClient
# uri = os.environ.get("MONGODB_URI")
# if not uri:
#     raise ValueError("Set MONGODB_URI in Colab Secrets or environment variables first.")
# client = MongoClient(uri, serverSelectionTimeoutMS=10000)
# client.admin.command("ping")
# db = client["bda_lab"]
# collection = db["sales"]
# collection.delete_many({})  # clears only this demo collection
# collection.insert_many(sales.assign(date=pd.to_datetime(sales["date"], format="%d-%m-%Y")).to_dict("records"))
# print("Inserted:", collection.count_documents({}))
# print("One document:", collection.find_one({}, {"_id": 0}))
# print("High revenue records:", list(collection.find({"revenue": {"$gt": 100000}}, {"_id": 0}).limit(5)))
# print("Group revenue by region:")
# pipeline = [{"$group": {"_id": "$region", "total_revenue": {"$sum": "$revenue"}, "avg_profit": {"$avg":
"$profit"}}}]
# for row in collection.aggregate(pipeline): print(row)
# collection.create_index("product")
# client.close()
```

B. HBase shell commands

HBase needs a running HBase service (typically on a configured Hadoop environment); a shell-only mock is not an HBase database.

create 'bda_sales', 'details'

put 'bda_sales', '1', 'details:product', 'Laptop'

put 'bda_sales', '1', 'details:region', 'North'

put 'bda_sales', '1', 'details:revenue', '12500.50'

put 'bda_sales', '2', 'details:product', 'Phone'

scan 'bda_sales'

get 'bda_sales', '1'

count 'bda_sales'

disable 'bda_sales'

enable 'bda_sales'

To remove table permanently:

disable 'bda_sales'

drop 'bda_sales'

HBase table design: `bda_sales` is the table, `details` is the column family, and `details:product` is a column qualifier. Use `put`, `get`, `scan`, `count`, `disable`, and `drop` in the HBase shell.

Experiment 4 — Hive database and descriptive statistics

Hive requires a Hive service and a table over accessible data. The following SQL is suitable for a Hive shell/beeline after the CSV is placed in HDFS.

CREATE DATABASE IF NOT EXISTS bda_lab;

USE bda_lab;

CREATE EXTERNAL TABLE IF NOT EXISTS sales (

record_id INT,

date STRING,

region STRING,

product STRING,

units_sold INT,

unit_price DOUBLE,

discount_rate DOUBLE,

revenue DOUBLE,

cost DOUBLE,

profit DOUBLE

)

ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'

WITH SERDEPROPERTIES ('separatorChar'=',', 'skip.header.line.count'='1')

STORED AS TEXTFILE

LOCATION '/user/hive/warehouse/bda_lab.db/sales';

SELECT COUNT(*) AS total_records,

AVG(revenue) AS avg_revenue,

MIN(revenue) AS min_revenue,

MAX(revenue) AS max_revenue,

STDDEV(revenue) AS revenue_stddev

FROM sales;

SELECT region, COUNT(*) AS records, SUM(revenue) AS total_revenue,

AVG(profit) AS average_profit

FROM sales GROUP BY region ORDER BY total_revenue DESC;

SELECT product, AVG(unit_price) AS avg_price, SUM(units_sold) AS units

FROM sales GROUP BY product ORDER BY units DESC;

Place the CSV at the stated HDFS location before querying. If Hive is not installed, use this Colab equivalent:

Code cell

```
print("Dataset descriptive statistics:")
display(sales[["units_sold","unit_price","discount_rate","revenue","cost","profit"]].describe().round(2))
print("\nRevenue by region:")
display(sales.groupby("region", as_index=False).agg(
    records=("record_id","count"), total_revenue=("revenue","sum"),
    average_profit=("profit","mean")).round(2))
print("\nSummary by product:")
display(sales.groupby("product", as_index=False).agg(
    avg_unit_price=("unit_price","mean"), total_units=("units_sold","sum"),
    total_revenue=("revenue","sum")).round(2))
```

Experiment 5 — Data Stream Algorithms

This section implements all three choices listed in the lab: DGIM, Bloom Filter, and Flajolet–Martin. For a submission, you can select one if your instructor permits.

A. DGIM algorithm (approximate count of 1s in a recent window)

This simple implementation stores buckets as `(size, end_timestamp)` and merges the two oldest buckets of a size when more than two exist.

Code cell

```
class DGIM:
    def __init__(self, window_size):
        self.N = window_size
        self.t = 0
        self.buckets = []  # newest first: [size, end time]
    def add(self, bit):
        self.t += 1
        self.buckets = [b for b in self.buckets if self.t - b[1] < self.N]
        if bit == 1:
            self.buckets.insert(0, [1, self.t])
            self._compress()
    def _compress(self):
        size = 1
        while True:
            indexes = [i for i,b in enumerate(self.buckets) if b[0] == size]
            if len(indexes) <= 2:
                size *= 2
                if size > self.N: break
                continue
            # merge the two oldest buckets of this size
            i1, i2 = indexes[-2], indexes[-1]
            end_time = self.buckets[i1][1]
            self.buckets[i1] = [size*2, end_time]
            del self.buckets[i2]
    def estimate_last(self, k):
        if k <= 0: return 0
        total, cutoff = 0, self.t-k
        for size, end_time in self.buckets:
            if end_time > cutoff:
                total += size
        if self.buckets:
            total -= self.buckets[-1][0] / 2
        return max(0, round(total))

stream = [1,0,1,1,0,1,0,0,1,1,0,1,1,1,0,1]
dgim = DGIM(window_size=16)
for bit in stream: dgim.add(bit)
k = 10
print("Stream:", stream)
print(f"DGIM estimate for last {k} bits:", dgim.estimate_last(k))
print("Exact count for comparison:", sum(stream[-k:]))
```

B. Bloom Filter (membership test with possible false positives)

Code cell

```
import hashlib
class BloomFilter:
    def __init__(self, size=500, hashes=4):
        self.size, self.hashes = size, hashes
        self.bits = bytearray(size)
    def _indexes(self, item):
        for i in range(self.hashes):
            digest = hashlib.sha256(f"{i}:{item}".encode()).digest()
            yield int.from_bytes(digest[:8], "big") % self.size
```

```
    def add(self, item):
        for idx in self._indexes(str(item)): self.bits[idx] = 1
    def might_contain(self, item):
        return all(self.bits[idx] for idx in self._indexes(str(item)))

bf = BloomFilter()
for item in ["Hadoop", "Hive", "MongoDB", "Spark"]: bf.add(item)
for item in ["Hadoop", "HBase", "Spark", "Pig"]:
    print(item, "might be present" if bf.might_contain(item) else "definitely absent")
print("Note: a positive result can be a false positive; a negative result is definitely absent.")
```

C. Flajolet–Martin distinct-count estimation (single-hash teaching version)

Code cell

```
def flajolet_martin(items):
    max_zeros = 0
    for item in items:
        h = int(hashlib.sha256(str(item).encode()).hexdigest()[:16], 16)
        if h == 0:
            zeros = 64
        else:
            zeros = (h & -h).bit_length() - 1  # trailing zero bits
        max_zeros = max(max_zeros, zeros)
    return 2 ** max_zeros

sample_stream = ["a","b","a","c","d","e","c","f","g","g","h"]
print("FM estimated distinct count:", flajolet_martin(sample_stream))
print("Exact distinct count:", len(set(sample_stream)))
print("A production FM implementation uses multiple hashes/registers to reduce variance.")
```

Experiment 6 — Social Network Analysis / Community Detection

A small example network is created here so the code runs without downloading data. Replace edges with a real edge-list CSV for your own dataset.

Code cell

```
%pip -q install networkx
```

Code cell

```
import networkx as nx
G = nx.Graph()
edges = [
    ("A","B"),("A","C"),("B","C"),("C","D"),
    ("D","E"),("D","F"),("E","F"),
    ("G","H"),("G","I"),("H","I"),("F","G")
]
G.add_edges_from(edges)
print("Nodes:", G.number_of_nodes(), "Edges:", G.number_of_edges())
print("Degree centrality:", {k: round(v,3) for k,v in nx.degree_centrality(G).items()})
print("Betweenness centrality:", {k: round(v,3) for k,v in nx.betweenness_centrality(G).items()})
try:
    from networkx.algorithms.community import greedy_modularity_communities
    communities = list(greedy_modularity_communities(G))
    for i, community in enumerate(communities, 1):
        print(f"Community {i}:", sorted(community))
except Exception as e:
    print("Community detection error:", e)
```

R alternative (if your lab specifically requires R)

Run in an R environment with `igraph` installed:

install.packages("igraph")

library(igraph)

edges <- data.frame(

from=c("A","A","B","C","D","D","E","G","G","H","F"),

to =c("B","C","C","D","E","F","F","H","I","I","G")

)

g <- graph_from_data_frame(edges, directed=FALSE)

print(degree(g))

print(betweenness(g))

communities <- cluster_fast_greedy(g)

print(communities)

plot(g, vertex.color=membership(communities), vertex.size=25,

vertex.label.color="black", main="Community Detection")

Experiment 7 — Data Visualization using Hive/PIG/R/Tableau

Here are runnable charts using the common dataset. For Hive, query aggregate tables first; export results to CSV, then chart them.

Code cell

```
%pip -q install matplotlib seaborn
```

Experiment 7 — Visualization code with dataset embedded

This code is self-contained: the eight sample records are written directly in the program. Run it in Colab without uploading a CSV.

```
# Experiment 7: Data Visualization (dataset included directly in the program)
# Run this entire cell in Google Colab. No CSV upload is needed.

import pandas as pd
import matplotlib.pyplot as plt

# Small sample dataset embedded directly in this program
data = {
    "Region": ["North", "South", "East", "West", "North", "South", "East", "West"],
    "Product": ["Laptop", "Phone", "Tablet", "Monitor", "Phone", "Laptop", "Monitor", "Tablet"],
    "Units_Sold": [12, 20, 15, 10, 18, 9, 14, 16],
    "Revenue": [72000.50, 45000.75, 22500.00, 18000.25, 40500.00, 54000.80, 25200.50, 24000.00],
    "Profit": [12000.25, 9000.50, 4500.00, 3200.75, 8100.00, 9500.25, 5100.00, 4700.50]
}

df = pd.DataFrame(data)
print("Dataset:")
display(df)

# 1. Bar chart: total revenue by region
region_revenue = df.groupby("Region", as_index=False)["Revenue"].sum()
plt.figure(figsize=(7, 4))
plt.bar(region_revenue["Region"], region_revenue["Revenue"])
plt.title("Total Revenue by Region")
plt.xlabel("Region")
plt.ylabel("Revenue")
plt.tight_layout()
plt.show()

# 2. Bar chart: units sold by product
product_units = df.groupby("Product", as_index=False)["Units_Sold"].sum()
plt.figure(figsize=(7, 4))
plt.bar(product_units["Product"], product_units["Units_Sold"])
plt.title("Units Sold by Product")
plt.xlabel("Product")
plt.ylabel("Units Sold")
plt.tight_layout()
plt.show()

# 3. Scatter plot: revenue vs profit
plt.figure(figsize=(7, 4))
plt.scatter(df["Revenue"], df["Profit"])
plt.title("Revenue vs Profit")
plt.xlabel("Revenue")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()

# 4. Print descriptive statistics
print("Descriptive statistics:")
display(df[["Units_Sold", "Revenue", "Profit"]].describe().round(2))
```

Hive aggregate example for visualization input

SELECT region, SUM(revenue) AS total_revenue

FROM bda_lab.sales GROUP BY region;

Export the query result to CSV and open it in Tableau, or read it in R/Python. Pig is a legacy ecosystem tool and needs a configured Hadoop/Pig environment; it is not normally preinstalled in Colab.

Experiment 8 — PySpark EDA and HITS algorithm

A. PySpark exploratory data analysis

Code cell

```
%pip -q install pyspark
```

Code cell

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, sum as spark_sum, count
spark = SparkSession.builder.master("local[*]").appName("BDA_Lab_EDA").getOrCreate()
spark_df = spark.read.csv("bda_sales_500.csv", header=True, inferSchema=True)
spark_df.printSchema()
spark_df.show(5, truncate=False)
spark_df.describe("units_sold","unit_price","revenue","profit").show()
spark_df.groupBy("region").agg(
    count("*").alias("records"),
    spark_sum("revenue").alias("total_revenue"),
    avg("profit").alias("average_profit")
).orderBy(col("total_revenue").desc()).show()
spark_df.groupBy("product").agg(
    spark_sum("units_sold").alias("units"),
    avg("unit_price").alias("average_price")
).show()
```

B. HITS algorithm (hubs and authorities) using NetworkX

Code cell

```
import networkx as nx
DG = nx.DiGraph()
DG.add_edges_from([
    ("pageA","pageC"),("pageB","pageC"),("pageB","pageD"),
    ("pageC","pageD"),("pageD","pageC"),("pageE","pageC"),
    ("pageE","pageD"),("pageA","pageD")
])
hubs, authorities = nx.hits(DG, max_iter=500, normalized=True)
print("Hub scores:")
for node, score in sorted(hubs.items(), key=lambda x: x[1], reverse=True):
    print(node, round(score, 4))
print("\nAuthority scores:")
for node, score in sorted(authorities.items(), key=lambda x: x[1], reverse=True):
    print(node, round(score, 4))
```

Experiment 9 — Mini-project: Sales Analytics and Recommendation

Dataset: `bda_sales_500.csv` generated in the first cell. For a submission, describe it honestly as a synthetic practice dataset. Your lab sheet also permits standard real-world datasets; if your instructor requires a public dataset, substitute a permitted source and cite its origin.

This mini-project performs trend analysis, region/product comparisons, and a simple product recommendation based on products that are frequently bought in the same region. This is a demonstration recommender, not a production personalization model.

Code cell

```
# 1. Data quality checks
print("Shape:", sales.shape)
print("Missing values:\n", sales.isna().sum())
print("Duplicate rows:", sales.duplicated().sum())

# 2. KPI summary
kpis = {
    "total_records": int(len(sales)),
```

```
    "total_revenue": round(float(sales["revenue"].sum()), 2),
    "total_profit": round(float(sales["profit"].sum()), 2),
    "average_order_revenue": round(float(sales["revenue"].mean()), 2),
    "total_units_sold": int(sales["units_sold"].sum())
}
print(kpis)

# 3. Top products and regions
display(sales.groupby("product", as_index=False).agg(
    total_revenue=("revenue","sum"), total_units=("units_sold","sum"),
    mean_profit=("profit","mean")
).sort_values("total_revenue", ascending=False).round(2))
display(sales.groupby("region", as_index=False).agg(
    total_revenue=("revenue","sum"), total_profit=("profit","sum")
).sort_values("total_revenue", ascending=False).round(2))

# 4. Simple recommendation: recommend the top-revenue product within a region
region_product = sales.groupby(["region","product"], as_index=False)["revenue"].sum()
def recommend_product(region_name):
    subset = region_product[region_product["region"].str.lower() == region_name.lower()]
    if subset.empty:
        return f"Unknown region: {region_name}. Choose North, South, East, or West."
    row = subset.sort_values("revenue", ascending=False).iloc[0]
    return f"Recommended product for {region_name}: {row['product']} (historical revenue {row['revenue']:.2f})"

for r in sorted(sales["region"].unique()):
    print(recommend_product(r))

# 5. Save summarized outputs
sales.groupby("region", as_index=False).agg(
    total_revenue=("revenue","sum"), total_profit=("profit","sum")
).to_csv("region_summary.csv", index=False)
sales.groupby("product", as_index=False).agg(
    total_revenue=("revenue","sum"), total_units=("units_sold","sum")
).to_csv("product_summary.csv", index=False)
print("Saved bda_sales_500.csv, region_summary.csv, product_summary.csv")
```

Results / conclusions template

1. HDFS commands demonstrate file management only when executed on a configured Hadoop cluster; the local demonstration is a fallback.

2. Word count shows the mapper/reducer pattern and the Hadoop Streaming files for cluster execution.

3. MongoDB commands demonstrate document insertion, filtering, aggregation, and indexing when a server is reachable; HBase shell commands require a running HBase service.

4. Hive SQL calculates count, average, minimum, maximum, standard deviation, and grouped summaries; the Pandas equivalent runs directly in Colab.

5. DGIM approximates recent-window counts, Bloom Filter supports memory-efficient membership tests, and Flajolet–Martin estimates distinct elements.

6. Network analysis calculates centrality and communities.

7. Charts compare revenue, profit, product sales, and trends.

8. PySpark demonstrates distributed-data APIs in local mode; HITS ranks hubs and authorities in a directed graph.

9. The mini-project demonstrates data cleaning, KPIs, grouping, visual analysis, and a simple recommendation rule.

Before submission: Add screenshots of actual outputs, your name/class details, and the exact dataset source if you replace the synthetic data. Do not claim a service ran unless you executed it.